WORK PLAN

Project: URL Shortener with Analytics Dashboard

Group: _7__ | Duration: 6 Weeks | Members: 4

No.	Objective	Activities / Tasks	Results / Deliverables	Who	When
1	Understand Rust fundamentals required for web development	1. Study Rust ownership, borrowing, and lifetimes 2. Learn structs, enums, pattern matching 3. Practice error handling with Result & Option 4. Set up development environment (VS Code + Rust toolchain) 5. Each member builds a small Rust CLI program	• Dev environment ready for all 4 members • Each member submits 1 working CLI program • Group agrees on coding conventions	All members	Week 1 (Days 1–7)
2	Learn Actix-web framework and set up project skeleton	1. Study HTTP basics: methods, status codes, headers 2. Learn Actix-web routing, handlers, JSON (serde) 3. Learn basic HTML, CSS, JS (fetch API) 4. Initialize Cargo workspace & project structure 5. Set up GitHub repository and GitHub Actions CI	• Running Actix-web server with mock endpoints • /shorten endpoint returns fake JSON response • GitHub repo with CI pipeline configured • Basic HTML form page created	Đặng Hoàng Tân: server & routing Nguyễn Trần Minh: HTML/JS/CSS Nguyễn Đức Long: DB schema design Ngô Duy Hoàng: GitHub repo & CI	Week 2 (Days 8–14)
3	Implement database layer and user authentication	1. Set up SQLite with SQLx and write DB migrations 2. Implement user registration and login endpoints 3. Integrate JWT token generation and validation 4. Hash passwords with bcrypt 5. Protect routes with JWT middleware 6. Build login/register HTML pages	• Users table created with migration • Register & login endpoints work correctly • JWT returned on login, validated on protected routes • Login/register UI functional • API tested via Postman / curl	Đặng Hoàng Tân: SQLx integration Nguyễn Trần Minh: Login/register UI Nguyễn Đức Long: JWT middleware & migrations Ngô Duy Hoàng: API testing	Week 3 (Days 15–21)
4	Build core feature: URL shortening and click tracking	1. Implement POST /shorten to create short codes 2. Implement GET /{code} redirect (HTTP 301) 3. Log every click: timestamp, IP, user agent 4. Implement GET /stats/{code} analytics endpoint 5. Build dashboard UI: list user's links 6. Write integration tests end-to-end	• Short URL creation works and persists to DB • Redirect to original URL works correctly • Click events recorded in database • Dashboard lists all links for logged-in user • End-to-end test cases passing	Đặng Hoàng Tân: shorten, redirect, click log Nguyễn Trần Minh: dashboard & link list UI Nguyễn Đức Long: analytics SQL queries Ngô Duy Hoàng: end-to-end testing & bug fixes	Week 4 (Days 22–28)
5	Finalize UI, add analytics charts, and deploy the application	1. Integrate Chart.js for click-over-time bar chart 2. Serve frontend as static files from Actix-web 3. Configure CORS headers correctly 4. Optimize DB queries and add indexes 5. Deploy to Railway or Render (free tier) 6. Write README and setup documentation	• Analytics dashboard shows bar chart of daily clicks • Full app served from single Rust binary • Live demo URL accessible online • README with setup instructions complete • Performance acceptable under basic load	Đặng Hoàng Tân: static file serving & CORS Nguyễn Trần Minh: Chart.js integration & UI polish Nguyễn Đức Long: DB optimization & indexes Ngô Duy Hoàng: deployment & README	Week 5 (Days 29–35)
6	Prepare and submit final report and presentation	1. Write technical report (architecture, design decisions) 2. Draw system architecture diagram 3. Prepare presentation slides 4. Conduct group rehearsal and refine demo script 5. Submit work plan file and final report 6. Perform live demo during defense	• Technical report (PDF) submitted on time • Architecture diagram included in report • Presentation slides ready • Live demo runs stably during defense • All group members can answer Q&A	Đặng Hoàng Tân: backend section of report Nguyễn Trần Minh: frontend section & UI screenshots Nguyễn Đức Long: DB & security section Ngô Duy Hoàng: compile report, slides & rehearsal lead	Week 6 (Days 36–42)